Bayes Filter and Kalman Filter

Adithi Pillai
Department of Computer Science and Engineering
Manipal Institute of Technology
Manipal, India
adithi.mitmpl2026@learner.manipal.edu

***Abstract*—This report explains how a driverless car combines an imperfect motion model with noisy sensors to estimate its own state. It derives the Bayes filter and defines every term in its predict and update steps, states the assumptions under which it reduces to the Kalman filter and why they make the mathematics tractable, and explains the Kalman filter equations one by one, including how the Kalman gain behaves for very noisy and very accurate sensors. It ends by locating the Kalman filter in the driverless (DV) pipeline of our car.**

Keywords—Bayes filter, Kalman filter, Kalman gain, state estimation, sensor fusion, GNSS/INS, extended Kalman filter

INTRODUCTION

A car never knows its true state. The wheel speed sensors, the VN-200 and the LiDAR each give a noisy view of where it is and how fast it is moving, and the motion model that predicts where it should be after a command is only approximate, because tyres slip and the road is not flat. Neither source can be trusted completely, but together they are far better than either alone. A filter is the rule for combining them. At every moment it keeps not a single guess but a probability distribution over the possible states, called the belief, and it updates that belief in two repeating steps: predict with the motion model, then correct with the newest measurement.

THE BAYES FILTER

Let x_t be the state of the car at time t (for example position, heading and speed), u_t the control applied between t−1 and t (steering and throttle commands), and z_t the measurement received at time t. The belief is the probability distribution of the state given everything known so far:

$$bel(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$$

Two Markov assumptions keep this manageable. Given the present state, the next state does not depend on older history, and a measurement depends only on the state at that instant. The filter therefore needs only the latest belief, not the whole history.

The two steps below follow directly from probability. Bayes' rule splits the belief into a measurement term and a term that holds only the past measurements. The Markov assumption reduces the measurement term to $p(z_t \mid x_t)$, and the other term is expanded by summing over the previous state, which produces the integral of the prediction step. The command u_t is known, so it only conditions the motion model.

Prediction step. The prior belief $bel(x_{t-1})$ is the belief carried over from the previous cycle. It is pushed through the motion model $p(x_t \mid x_{t-1}, u_t)$, which gives the probability of each new state if the car was at x_{t-1} and received command u_t. By the law of total probability:

$$\overline{bel}(x_t) = \int p(x_t \mid x_{t-1}, u_t)\, bel(x_{t-1})\, dx_{t-1}$$

The integral sums over every state the car might have been in, each weighted by how believable it was. The result, written with a bar, is the predicted belief: the prior for time t, before the measurement is used. Because the motion is noisy, prediction always spreads the belief out, so uncertainty grows.

Update step. The measurement likelihood $p(z_t \mid x_t)$ is the probability of obtaining the reading z_t if the true state were x_t. It encodes the sensor model and its noise. Bayes' rule combines it with the predicted belief:

$$bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)$$

States that explain the measurement well are boosted and states that do not are suppressed. The product on its own does not integrate to one, so it is not yet a valid probability distribution. The normaliser η rescales it:

$$\eta = 1 / p(z_t \mid z_{1:t-1}, u_{1:t}) = 1 / \int p(z_t \mid x_t)\, \overline{bel}(x_t)\, dx_t$$

The denominator is the total probability of seeing this measurement under every possible state. It does not depend on x_t, so it is just a constant, and its only job is to make the belief sum to one. The update step narrows the belief, so uncertainty shrinks. The corrected belief becomes the prior of the next cycle. In general the two integrals have no closed form, which is why practical filters approximate them with grids, particles or, as below, Gaussians.

FROM BAYES FILTER TO KALMAN FILTER

The Kalman filter is the Bayes filter under three assumptions. First, the motion model is linear with additive Gaussian noise:

$$x_t = A\, x_{t-1} + B\, u_t + w_t, \quad w_t \sim N(0, Q)$$

Second, the measurement model is linear with additive Gaussian noise:

$$z_t = H\, x_t + v_t, \quad v_t \sim N(0, R)$$

Third, the initial belief is Gaussian, $bel(x_0) = N(\mu_0, \Sigma_0)$. The noises w_t and v_t are white, meaning independent from one step to the next and from each other.

These assumptions make the mathematics tractable because a Gaussian is completely described by two quantities, its mean and its covariance, and three properties keep it Gaussian. A linear function of a Gaussian is Gaussian: if $x \sim N(\mu, \Sigma)$, then $Ax \sim N(A\mu, A\Sigma A^T)$. The sum of independent Gaussians is Gaussian with the covariances added, which is exactly what the prediction integral computes, since it blends the belief with the motion noise. The product of two Gaussians is, up to a constant, another Gaussian, which is exactly what the update computes, and the constant is absorbed by η. The belief therefore stays Gaussian forever, and instead of integrating over a whole function the filter only updates a mean and a covariance with a few matrix operations. The cost grows polynomially with the state size,

whereas a grid over the state grows exponentially. Under these assumptions the result is also optimal, giving the minimum mean-square error estimate.

When the assumptions fail, the filter breaks in recognisable ways. With nonlinear models, such as a car whose position depends on cos and sin of its heading, a Gaussian passed through the function is no longer Gaussian, so the linear formulas give a biased mean and a wrong covariance. The extended Kalman filter (EKF) linearises the model with Jacobians and the unscented Kalman filter (UKF) uses sigma points, but both are approximations that can diverge if the nonlinearity is strong or the starting guess is poor. With non-Gaussian noise, such as GNSS multipath outliers or the one-sided error of wheel slip, the filter treats a bad reading as an ordinary one and is dragged away from the truth, so gating or robust methods are needed. With a multi-modal belief, such as two possible cone assignments, a single Gaussian has one peak and cannot hold two hypotheses, and a particle filter is needed. Finally, wrong values of Q and R make the filter overconfident, so it ignores good measurements, or twitchy, so it follows every noisy one.

THE KALMAN FILTER

The filter stores the belief as an estimate $\hat{x}$ (the mean) and a covariance P (the uncertainty). In $\hat{x}_{k|k-1}$, the subscript means the estimate for step k using measurements up to step k−1. Each cycle has two phases.

Predict. The state estimate is moved forward with the motion model and the covariance is grown by the process noise:

$$\hat{x}_{k|k-1} = A\,\hat{x}_{k-1|k-1} + B\,u_k$$
$$P_{k|k-1} = A\,P_{k-1|k-1}\,A^T + Q$$

The first equation says where the car should be if the model were perfect. In the second, $A\,P\,A^T$ shows how the old uncertainty is stretched by the dynamics, and Q adds the new uncertainty of the model itself. P can only grow here.

Update. The innovation is the surprise, the difference between what the sensor measured and what the prediction expected it to measure. Its covariance S says how large a surprise is normal:

$$y_k = z_k - H\,\hat{x}_{k|k-1}$$
$$S_k = H\,P_{k|k-1}\,H^T + R$$

The Kalman gain decides how much of the innovation to believe:

$$K_k = P_{k|k-1}\,H^T\,S_k^{-1}$$

It is then used to correct the estimate and shrink the uncertainty:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k\,y_k$$
$$P_{k|k} = (I - K_k\,H)\,P_{k|k-1}$$

The corrected estimate is the prediction plus a gain-weighted share of the surprise. The new covariance is never larger than the predicted one, since every measurement adds information. In software the Joseph form of this covariance equation is often used because it keeps P symmetric and positive definite despite rounding.

Two of these equations deserve a closer look. S has two parts: $H\,P\,H^T$ is the prediction's own uncertainty seen in measurement units, and R is the sensor noise. They simply add, because the prediction error and the measurement error are independent. The gain $K = P\,H^T\,S^{-1}$ is the choice that minimises the total variance left after the update. Read it as the uncertainty of the state, seen through H, divided by the total uncertainty of the innovation. H^T and the inverse convert between state units and measurement units, so the correction is applied to the right states in the right amounts.

The gain is clearest for one variable with H = 1, where $K = P_{k|k-1} / (P_{k|k-1} + R)$. It is the share of the total uncertainty that belongs to the model. Take a predicted position of 10 m with $P_{k|k-1} = 4$ m² and a GNSS reading of 12 m. If the sensor is accurate, R = 1 m², then K = 0.8, the estimate moves to 11.6 m and the new variance is 0.8 m². If it is very noisy, R = 16 m², then K = 0.2, the estimate moves only to 10.4 m and the variance is 3.2 m². As R grows without limit, K tends to 0, the measurement is ignored and the filter coasts on the model, so the uncertainty stays at its predicted value. As R tends to 0, K tends to 1 (in general to H^{-1}), the estimate snaps to the measurement and the uncertainty collapses. A larger Q works the other way round, because it inflates $P_{k|k-1}$ and pushes K towards 1. Because P also holds correlations between states, a position measurement can correct the velocity estimate as well, through the off-diagonal terms of K.

Running the loop for a second cycle shows the two phases working against each other. Continue from the corrected values $\hat{x}$ = 11.6 m and P = 0.8 m², with A = 1, a commanded advance of B u = 2 m, process noise Q = 0.5 m² and a new reading z = 13.0 m with R = 1 m². The prediction gives $\hat{x}_{k|k-1}$ = 13.6 m and $P_{k|k-1} = 0.8 + 0.5 = 1.3$ m², so the uncertainty has grown. The innovation is $y = 13.0 - 13.6 = -0.6$ m, and the gain is $K = 1.3 / 2.3 = 0.565$. The corrected estimate is $13.6 - 0.565 \times 0.6 = 13.26$ m and the new variance is $(1 - 0.565) \times 1.3 = 0.565$ m². Across the cycles the variance went from 4 to 0.8 to 0.57. Each prediction raises it by Q, each update lowers it by more, and it settles at a steady value set by the balance between Q and R.

RELEVANCE TO THE CAR

The main home of the Kalman filter in the DV pipeline is localisation, which is estimating the car's own pose and speed. The state would hold position, heading, speed and the gyro bias. The prediction runs at the IMU rate: it is fast and smooth, but it drifts because it integrates. The corrections arrive more slowly: GNSS position, which is noisy and can vanish near buildings, wheel speed $v = \omega r$, and the pose from LiDAR scan matching. This is exactly the structure of the VN-200 which runs such a filter internally, so downstream we would run a second filter that fuses its output with wheel odometry and the LiDAR pose. Heading makes the model nonlinear, so in practice this is an EKF or UKF. The covariance P is as valuable as the estimate: when GNSS drops out, P grows at every prediction, and the controller can use that growth to refuse to steer blindly.

The second place is cone tracking and mapping. Each cone gets its own small filter with the state (x, y) of a stationary landmark. The prediction is trivial, with A = I and a tiny Q (or a shift by the car's own motion), and each LiDAR cluster centroid is an update with an R that grows with range. The innovation covariance S also gives a gate: a detection is assigned to an existing cone only if $y^T\,S^{-1}\,y$ is below a threshold, which handles data association. Repeated sightings shrink P, so the cones of the map become more precise on every lap.

The third is fusing the LiDAR and the ZED 2i. Both measure the distance to a cone, but the LiDAR has a small,

nearly constant R, while the stereo depth error grows roughly with the square of the distance. The gain then weights each sensor automatically and replaces a hard rule of trusting the LiDAR beyond a fixed range. An innovation that is large compared with S, meaning that the two disagree, becomes the warning the camera section of the report asks for. The same idea detects wheel slip, because the wheel measurement's innovation against the IMU grows, and R for the wheels can then be inflated. Since a Kalman filter simply predicts up to each measurement's timestamp before updating, sensors with different rates and delays can be fused cleanly.

CONCLUSION

The Bayes filter is the general recipe: predict with the motion model, then correct with the measurement likelihood and normalise. The Kalman filter is the special case in which linear models and Gaussian noise reduce that recipe to a few matrix equations, with the Kalman gain balancing the model against the sensor according to their uncertainties. Where those assumptions fail, EKF, UKF or particle filters extend the idea. In our car the Kalman filter sits at the heart of localisation, cone tracking and sensor fusion.

REFERENCES

[1] J. W. Lambert, "Bayes Filter." [Online]. Available: https://johnwlambert.github.io/bayes-filter/

[2] KalmanFilter.NET, "Kalman Filter Tutorial." [Online]. Available: https://kalmanfilter.net/